

CIS 5357

Computing for Data Analytics

Week 12

Last week

- Editing dataframes in pandas
- Approaches to messy & missing data

Quick review

Editing DataFrames in pandas

- Add columns with square brackets *df_name["new_column"]*
- Add rows with *concat()*
- Remove columns or rows with *drop()*
- Edit columns by overwriting
- Use *loc()*, *apply()*, *map()* or *replace()* to edit specific values

Address messy data

- Missing data: *isnull()*; *fillna()*
- Evaluate outliers

Practice with census.gov data

- Load data
- Filter for NY records
- Estimate future growth
- How would you join this to Dig?

Practice with census.gov data

- **Load data**
- Filter for NY records
- Estimate future growth
- How would you join this to Dig?

```
#importing the pandas library
import pandas as pd

#loading the file
df = pd.read_csv('co-est2022-alldata.csv')

df.info()

✓ 0.0s
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 3195 entries, 0 to 3194

Data columns (total 51 columns):

#	Column	Non-Null Count	Dtype
0	SUMLEV	3195 non-null	int64
1	REGION	3195 non-null	int64
2	DIVISION	3195 non-null	int64
3	STATE	3195 non-null	int64
4	COUNTY	3195 non-null	int64
5	STNAME	3195 non-null	object
6	CTYNAME	3195 non-null	object
7	ESTIMATESBASE2020	3195 non-null	int64
8	POPESTIMATE2020	3195 non-null	int64
9	POPESTIMATE2021	3195 non-null	int64
10	POPESTIMATE2022	3195 non-null	int64
11	NPOPCHG2020	3195 non-null	int64
12	NPOPCHG2021	3195 non-null	int64
13	NPOPCHG2022	3195 non-null	int64

Practice with census.gov data

- Load data
- **Filter for NY records**

```
#Filter for just the New York state records
ny_df = df[(df.STNAME == 'New York') & (df.SUMLEV == 50)]

#Keep just the relevant columns
ny_df = ny_df[['CTYNAME', 'ESTIMATESBASE2020',
               'POPESTIMATE2020', 'POPESTIMATE2021', 'POPESTIMATE2022',
               'NPOPCHG2020', 'NPOPCHG2021', 'NPOPCHG2022']]

#Look for missing values
ny_df.isnull().value_counts()
```

✓ 0.0s

CTYNAME	ESTIMATESBASE2020	POPESTIMATE2020	POPESTIMATE2021	POPESTIMATE2022	NPOPCHG2020	NPOPCHG2021	NPOPCHG2022	
False	False	False	False	False	False	False	False	62
dtype: int64								

Practice with census.gov data

- Load data
- **Filter for NY records**

```
#Calculate statistics  
ny_df.describe()
```

✓ 0.0s

	ESTIMATESBASE2020	POPESTIMATE2020	POPESTIMATE2021	POPESTIMATE2022	NPOPCHG2020	NPOPCHG2021	NPOPCHG2022
count	6.200000e+01	6.200000e+01	6.200000e+01	6.200000e+01	62.000000	62.000000	62.000000
mean	3.258263e+05	3.243274e+05	3.202821e+05	3.173734e+05	-1498.935484	-4045.225806	-2908.725806
std	5.713474e+05	5.677553e+05	5.537871e+05	5.462171e+05	3969.019662	18480.867492	10295.709317
min	5.102000e+03	5.073000e+03	5.114000e+03	5.118000e+03	-17031.000000	-98505.000000	-50112.000000
25%	4.872725e+04	4.863525e+04	4.877250e+04	4.845150e+04	-533.250000	-409.250000	-1082.750000
50%	8.771900e+04	8.756900e+04	8.701550e+04	8.627550e+04	-119.000000	-70.000000	-389.500000
75%	2.346520e+05	2.347392e+05	2.359570e+05	2.363092e+05	-50.750000	580.000000	-131.750000
max	2.736075e+06	2.719044e+06	2.637486e+06	2.590516e+06	1257.000000	11168.000000	17472.000000

Practice with census.gov data

- Load data
- Filter for NY records
- **Estimate future growth**

```
#Extrapolate recent growth rates using the average annual percent change in population from 2021-2022
ny_df['GROWTH_RATE'] = ((ny_df['POPESTIMATE2022'] - ny_df['POPESTIMATE2021']) / ny_df['POPESTIMATE2021']) * 100
ny_df['ESTIMATE_2023'] = ny_df['POPESTIMATE2022'] + (ny_df['GROWTH_RATE'] * ny_df['POPESTIMATE2022'] / 100)
ny_df['ESTIMATE_2023'] = ny_df['ESTIMATE_2023'].astype(int)
ny_df.head()
```

✓ 0.0s

Python

STIMATESBASE2020	POPESTIMATE2020	POPESTIMATE2021	POPESTIMATE2022	NPOPCHG2020	NPOPCHG2021	NPOPCHG2022	GROWTH_RATE	ESTIMATE_2023
314838	313987	316301	315811	-851	2314	-490	-0.154916	315321
46446	47320	46921	46694	874	-399	-227	-0.483792	46468
1472656	1461125	1421089	1379946	-11531	-40036	-41143	-2.895174	1339994
198675	198126	198703	197117	-549	577	-1586	-0.798176	195543
77041	76992	76650	76439	-49	-342	-211	-0.275277	76228

Practice with census.gov data

- Load data
- Filter for NY records
- Estimate future growth
- **How would you join this to Dig?**

This week

Chapter 7: BRINGING TOGETHER DATASETS

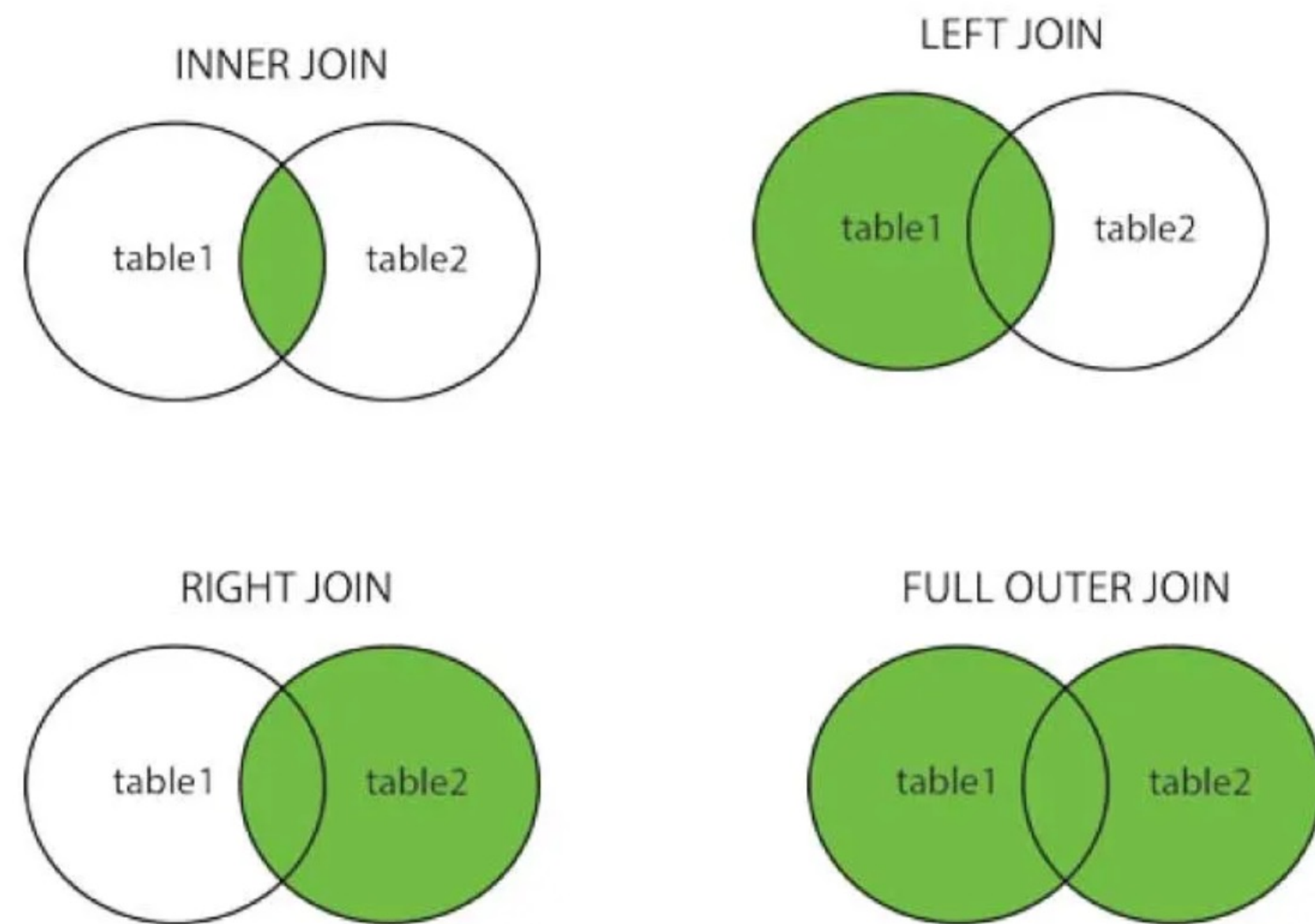
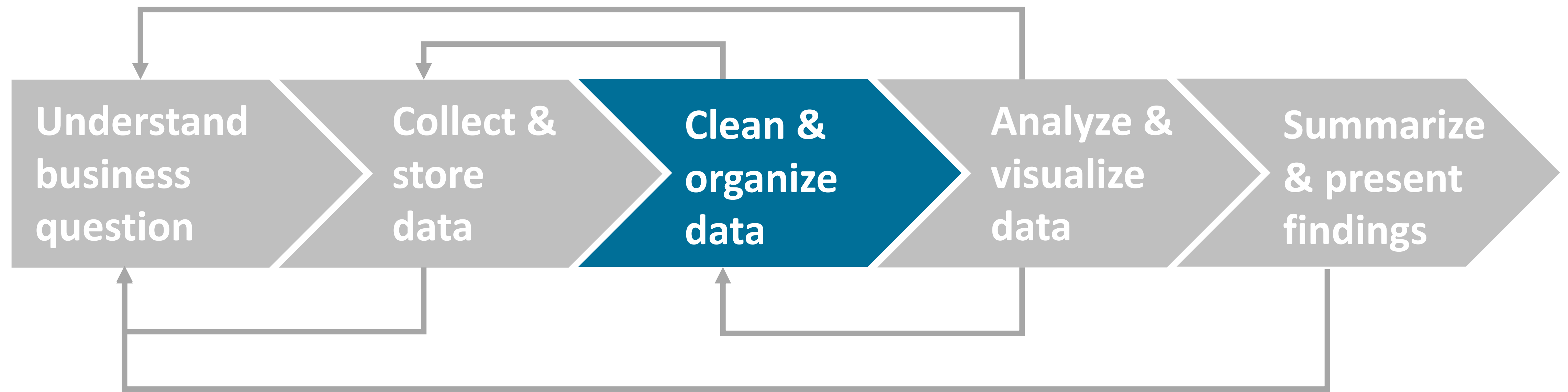


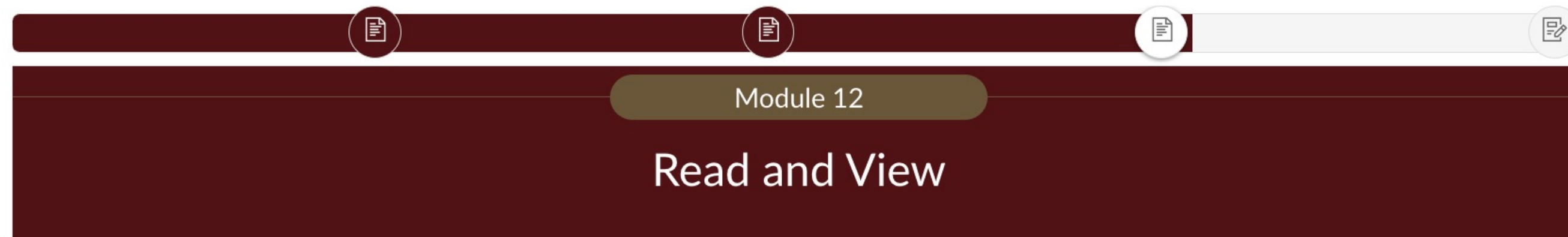
Image source — Google

Data Analytics

the process of analyzing raw data in order to draw out meaningful, actionable insights, which are then used to inform and drive business decisions

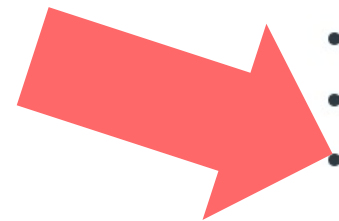


Download code to follow along



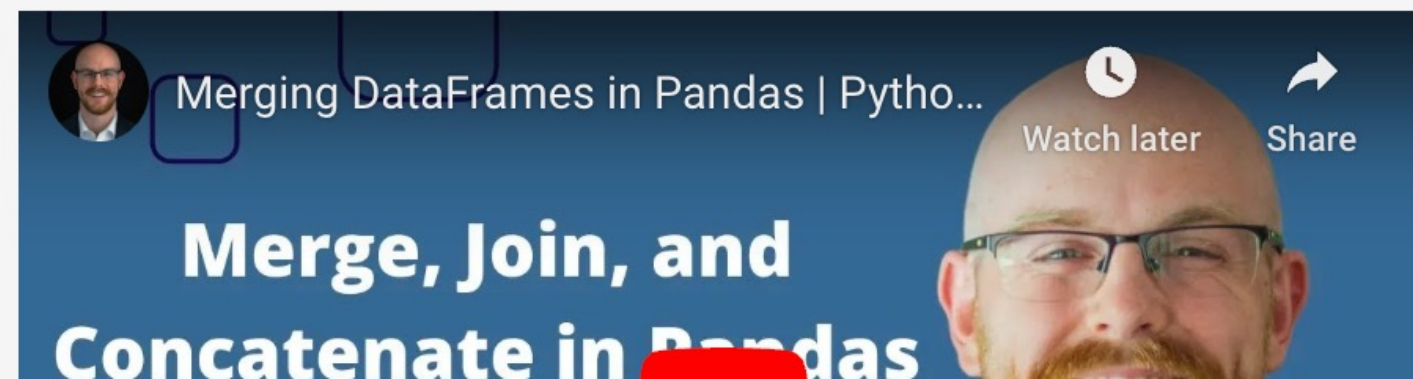
Read

- Textbook chapter 7
- [Pandas: merge and join](#) ↗
- [merge function documentation](#) ↗
- [drop_duplicates documentation](#) ↗
- [Lecture practice code](#) ↓



View

[Merging dataframes in pandas](#) ↗



5 types of joins

1. union
2. inner
3. outer
4. left
5. right

Union overview

- A method for **bringing two datasets together into one longer table**/dataframe; typically one table is added as additional rows to another table.
- Useful if your **files are split** into a few files due to size or maybe you **have new dataset** with the latest information that you want to append to an existing dataset.
- To perform a union in pandas, simply use the **pd.concat()** function, and pass a list of the DataFrames you want to bring together.

Union example

dataframe: **df_mascots_1**

	HOGWARTS_HOUSE	ANIMAL
0	Gryffindor	Lion
1	Slytherin	Snake

dataframe: **df_mascots_2**

	HOGWARTS_HOUSE	ANIMAL
0	Ravenclaw	Eagle
1	Hufflepuff	Badger

note the square brackets here, as the input needs to be a list

```
✓ df_mascots = pd.concat([df_mascots_1, df_mascots_2])  
df_mascots  
] ✓ 0.0s
```

	HOGWARTS_HOUSE	ANIMAL
0	Gryffindor	Lion
1	Slytherin	Snake
0	Ravenclaw	Eagle
1	Hufflepuff	Badger

Union example

dataframe: **df_mascots_1**

	HOGWARTS_HOUSE	ANIMAL
0	Gryffindor	Lion
1	Slytherin	Snake

dataframe: **df_mascots_2**

	HOGWARTS_HOUSE	ANIMAL
0	Ravenclaw	Eagle
1	Hufflepuff	Badger

```
✓ df_mascots = pd.concat([df_mascots_1, df_mascots_2])  
df_mascots  
] ✓ 0.0s
```

	HOGWARTS_HOUSE	ANIMAL
0	Gryffindor	Lion
1	Slytherin	Snake
0	Ravenclaw	Eagle
1	Hufflepuff	Badger

Union example – reset index

use **reset_index()** to avoid duplicates

```
#let's reset the index to avoid duplicates  
df_mascots = pd.concat([df_mascots_1, df_mascots_2]).reset_index()  
df_mascots
```

✓ 0.0s

	index	HOGWARTS_HOUSE	ANIMAL	SOURCE
0	0	Gryffindor	Lion	mascots_1
1	1	Slytherin	Snake	mascots_1
2	0	Ravenclaw	Eagle	mascots_2
3	1	Hufflepuff	Badger	mascots_2

Union example – drop original index

use **reset_index(drop=True)** to drop the original index

```
#if you didn't care to keep the original index use drop=True argument with reset_index() function  
df_mascots = pd.concat([df_mascots_1, df_mascots_2]).reset_index(drop=True)  
df_mascots
```

✓ 0.0s

	HOGWARTS_HOUSE	ANIMAL
0	Gryffindor	Lion
1	Slytherin	Snake
2	Ravenclaw	Eagle
3	Hufflepuff	Badger

Union example – drop original index

ignore_index=True also works

```
#ignore_index=True also works  
df_mascots = pd.concat([df_mascots_1, df_mascots_2], ignore_index=True)  
df_mascots
```

✓ 0.0s

	HOGWARTS_HOUSE	ANIMAL
0	Gryffindor	Lion
1	Slytherin	Snake
2	Ravenclaw	Eagle
3	Hufflepuff	Badger

Union example – retain the source

create a new column to store this information

```
#if you wanted to track which record came from which dataset, you could add a column for this
df_mascots_1['SOURCE'] = 'mascots_1'
df_mascots_2['SOURCE'] = 'mascots_2'
df_mascots = pd.concat([df_mascots_1, df_mascots_2]).reset_index(drop=True)
df_mascots
```

✓ 0.0s

	HOGWARTS_HOUSE	ANIMAL	SOURCE
0	Gryffindor	Lion	mascots_1
1	Slytherin	Snake	mascots_1
2	Ravenclaw	Eagle	mascots_2
3	Hufflepuff	Badger	mascots_2

Union example – retain the source

create keys for each data source

```
#can also create keys for each data source
df_mascots = pd.concat([df_mascots_1, df_mascots_2], keys = ['mascots_1', 'mascots_2'])
df_mascots
```

✓ 0.0s

		HOGWARTS_HOUSE	ANIMAL	SOURCE
mascots_1	0	Gryffindor	Lion	mascots_1
	1	Slytherin	Snake	mascots_1
mascots_2	0	Ravenclaw	Eagle	mascots_2
	1	Hufflepuff	Badger	mascots_2

Union example – access a key

Use **.loc()** to access specific keys

```
# with keys you can retrieve the data of a specific DataFrame using the loc method  
df_mascots.loc['mascots_2']
```

✓ 0.0s

	HOGWARTS_HOUSE	ANIMAL	SOURCE
0	Ravenclaw	Eagle	mascots_2
1	Hufflepuff	Badger	mascots_2

Union example – with a dictionary

pass a dictionary & those keys will be used for the keys argument

```
# can also pass a dictionary to concat(), in which case the dictionary keys will be used for the keys argument
data = {'mascots_1': df_mascots_1, 'mascots_2': df_mascots_2}
df_mascots = pd.concat(data)
df_mascots
```

] ✓ 0.0s

		HOGWARTS_HOUSE	ANIMAL	SOURCE
mascots_1	0	Gryffindor	Lion	mascots_1
	1	Slytherin	Snake	mascots_1
mascots_2	0	Ravenclaw	Eagle	mascots_2
	1	Hufflepuff	Badger	mascots_2

Concat along columns

To join DataFrames along column, specify the axis parameter as 1

```
# to join DataFrames along column, specify the axis parameter as 1
df_mascots = pd.concat([df_mascots_1, df_mascots_2], axis=1)
df_mascots
```

✓ 0.0s

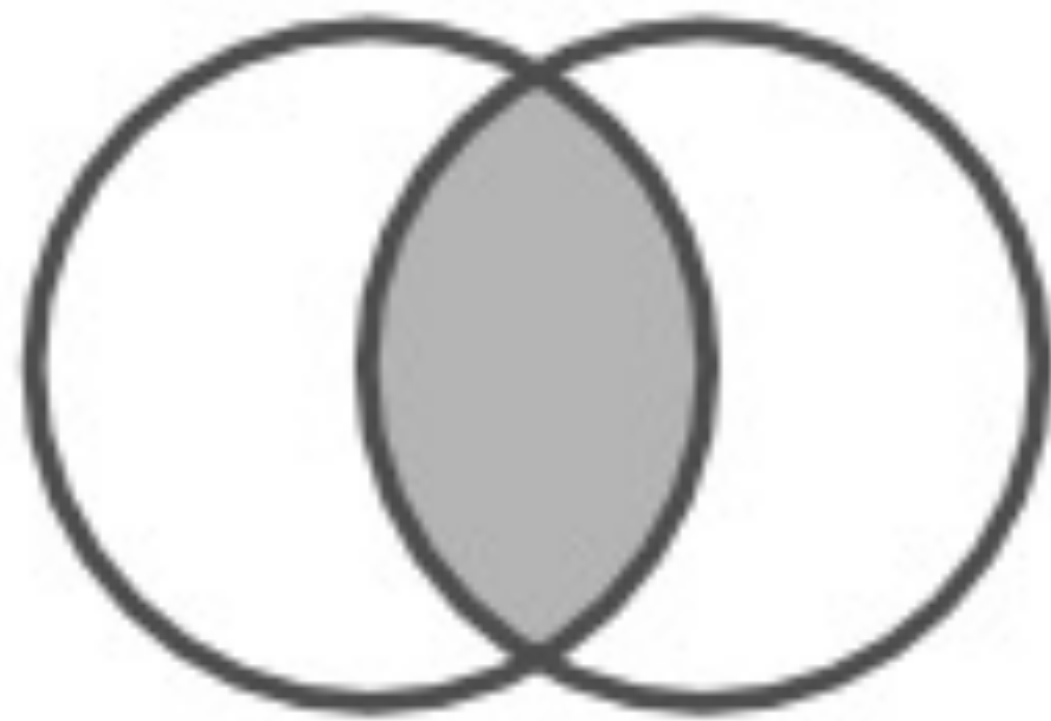
	HOGWARTS_HOUSE	ANIMAL	SOURCE	HOGWARTS_HOUSE	ANIMAL	SOURCE
0	Gryffindor	Lion	mascots_1	Ravenclaw	Eagle	mascots_2
1	Slytherin	Snake	mascots_1	Hufflepuff	Badger	mascots_2

In pandas axis = 0 refers to horizontal axis or rows and axis = 1 refers to vertical axis or columns

the above doesn't really make sense for this data, so let's recreate the mascots dataframe in our code file

Inner join

Include only those rows that **appear in both tables**—if a row is missing from one table or the other, drop it



Inner join example

df_hp_students

	NAME	HOUSE
0	Harry Potter	Gryffindor
1	Ron Weasley	Gryffindor
2	Luna Lovegood	Ravenclaw

df_mascots

	HOGWARTS_HOUSE	ANIMAL	SOURCE
0	Gryffindor	Lion	mascots_1
1	Slytherin	Snake	mascots_1
2	Ravenclaw	Eagle	mascots_2
3	Hufflepuff	Badger	mascots_2

“left” table

“right” table

```
pd.merge(df_hp_students,  
df_mascots,  
left_on='HOUSE',  
right_on='HOGWARTS_HOUSE',  
how='inner')
```

} fields to join on

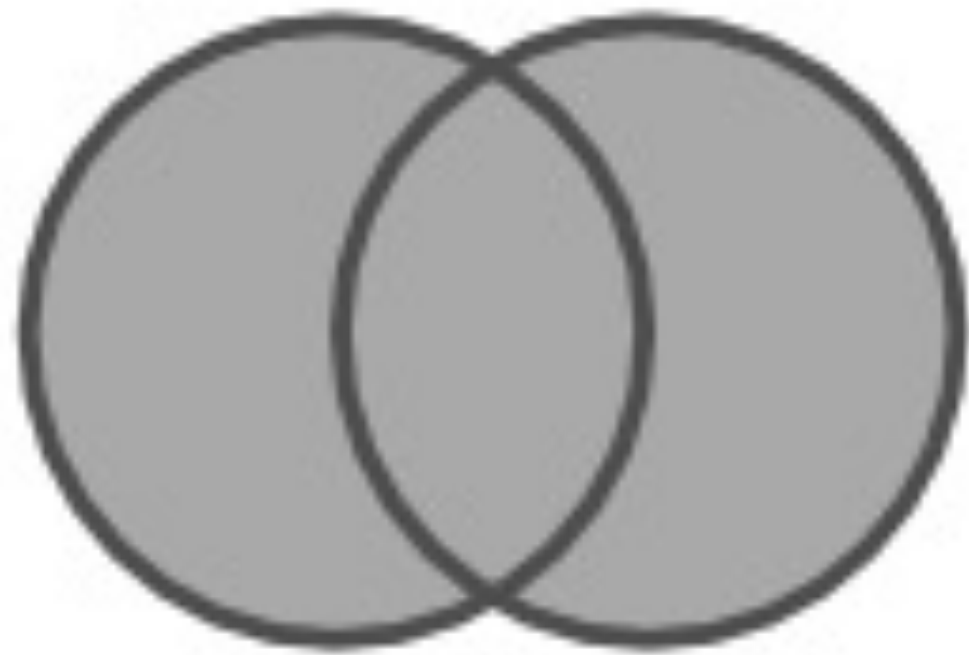
join type

✓ 0.0s

	NAME	HOUSE	HOGWARTS_HOUSE	ANIMAL	SOURCE
0	Harry Potter	Gryffindor	Gryffindor	Lion	mascots_1
1	Ron Weasley	Gryffindor	Gryffindor	Lion	mascots_1
2	Luna Lovegood	Ravenclaw	Ravenclaw	Eagle	mascots_2

Outer join

Include **all rows**, regardless of whether they appear in one table, the other, or both.



Outer join example

df_hp_students

	NAME	HOUSE
0	Harry Potter	Gryffindor
1	Ron Weasley	Gryffindor
2	Luna Lovegood	Ravenclaw

df_mascots

	HOGWARTS_HOUSE	ANIMAL	SOURCE
0	Gryffindor	Lion	mascots_1
1	Slytherin	Snake	mascots_1
2	Ravenclaw	Eagle	mascots_2
3	Hufflepuff	Badger	mascots_2

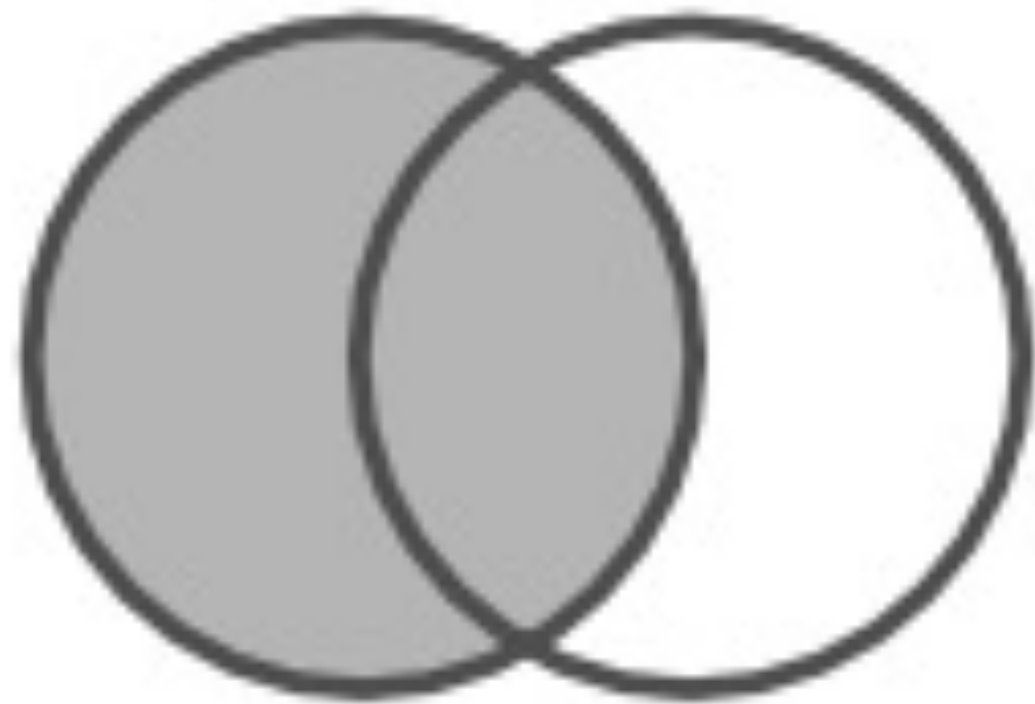
```
pd.merge(df_hp_students,
        df_mascots,
        left_on='HOUSE',
        right_on='HOGWARTS_HOUSE',
        how='outer')
```

✓ 0.0s

	NAME	HOUSE	HOGWARTS_HOUSE	ANIMAL	SOURCE
0	Harry Potter	Gryffindor	Gryffindor	Lion	mascots_1
1	Ron Weasley	Gryffindor	Gryffindor	Lion	mascots_1
2	Luna Lovegood	Ravenclaw	Ravenclaw	Eagle	mascots_2
3	NaN	NaN	Slytherin	Snake	mascots_1
4	NaN	NaN	Hufflepuff	Badger	mascots_2

Left join

A left join is one in which we **keep all the rows in the “left table”** and brings in rows from the right dataset if they exist.



Left join example

df_hp_students

	NAME	HOUSE
0	Harry Potter	Gryffindor
1	Ron Weasley	Gryffindor
2	Luna Lovegood	Ravenclaw

df_mascots

	HOGWARTS_HOUSE	ANIMAL	SOURCE
0	Gryffindor	Lion	mascots_1
1	Slytherin	Snake	mascots_1
2	Ravenclaw	Eagle	mascots_2
3	Hufflepuff	Badger	mascots_2

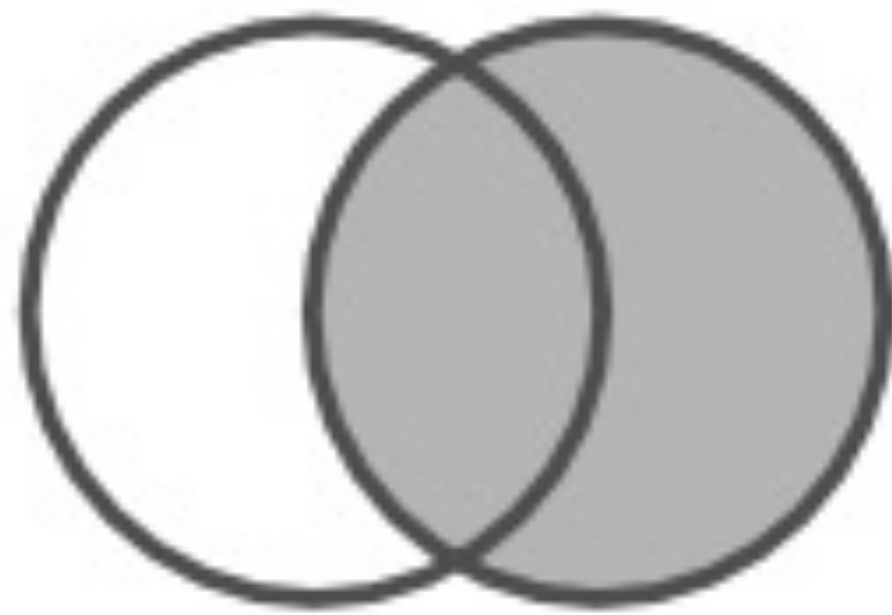
```
pd.merge(df_hp_students,  
        df_mascots,  
        left_on='HOUSE',  
        right_on='HOGWARTS_HOUSE',  
        how='left')
```

✓ 0.0s

	NAME	HOUSE	HOGWARTS_HOUSE	ANIMAL	SOURCE
0	Harry Potter	Gryffindor	Gryffindor	Lion	mascots_1
1	Ron Weasley	Gryffindor	Gryffindor	Lion	mascots_1
2	Luna Lovegood	Ravenclaw	Ravenclaw	Eagle	mascots_2

Right join

A right join is one in which we **keep all the rows in the “right table”** and brings in rows from the left dataset if they exist.



Right join example

df_hp_students

	NAME	HOUSE
0	Harry Potter	Gryffindor
1	Ron Weasley	Gryffindor
2	Luna Lovegood	Ravenclaw

df_mascots

	HOGWARTS_HOUSE	ANIMAL	SOURCE
0	Gryffindor	Lion	mascots_1
1	Slytherin	Snake	mascots_1
2	Ravenclaw	Eagle	mascots_2
3	Hufflepuff	Badger	mascots_2

```
pd.merge(df_hp_students,  
        df_mascots,  
        left_on='HOUSE',  
        right_on='HOGWARTS_HOUSE',  
        how='right')
```

✓ 0.0s

	NAME	HOUSE	HOGWARTS_HOUSE	ANIMAL	SOURCE
0	Harry Potter	Gryffindor	Gryffindor	Lion	mascots_1
1	Ron Weasley	Gryffindor	Gryffindor	Lion	mascots_1
2	NaN	NaN	Slytherin	Snake	mascots_1
3	Luna Lovegood	Ravenclaw	Ravenclaw	Eagle	mascots_2
4	NaN	NaN	Hufflepuff	Badger	mascots_2

Select specific columns

merge() will **include every column in both tables**, but you can select just specific columns in the tables before combining them (note both join keys need to be present)

```
✓ # choose specific columns to include in the merge
pd.merge(df_hp_students[['NAME', 'HOUSE']],
df_mascots[['HOGWARTS_HOUSE', 'ANIMAL']],
left_on='HOUSE',
right_on='HOGWARTS_HOUSE',
how='left')
] ✓ 0.0s
```

	NAME	HOUSE	HOGWARTS_HOUSE	ANIMAL
0	Harry Potter	Gryffindor	Gryffindor	Lion
1	Ron Weasley	Gryffindor	Gryffindor	Lion
2	Luna Lovegood	Ravenclaw	Ravenclaw	Eagle

We could also drop columns

```
# or we could also use the drop method
pd.merge(df_hp_students,
        df_mascots,
        left_on='HOUSE',
        right_on='HOGWARTS HOUSE',
        how='left').drop(columns=['HOUSE', 'SOURCE'])
```

1 ✓ 0.0s

	NAME	HOGWARTS_HOUSE	ANIMAL
0	Harry Potter	Gryffindor	Lion
1	Ron Weasley	Gryffindor	Lion
2	Luna Lovegood	Ravenclaw	Eagle

Using the *on* argument

If the name of the columns containing the join key is the same in both tables, we can use the *on* argument instead of specifying both *left_on* and *right_on*.

```
✓ # use the on argument when the join columns are the same name
df_mascots2= df_mascots.rename(columns={'HOGWARTS_HOUSE': 'HOUSE'})
pd.merge(df_hp_students,
        df_mascots2,
        on = 'HOUSE',
        how='left')
] ✓ 0.0s
```

	NAME	HOUSE	ANIMAL	SOURCE
0	Harry Potter	Gryffindor	Lion	mascots_1
1	Ron Weasley	Gryffindor	Lion	mascots_1
2	Luna Lovegood	Ravenclaw	Eagle	mascots_2

Using the *validate* argument

To check if the join keys are of unique values, you can use the *validate* argument.

- *one_to_one* will ensure that the join keys in **both tables are unique**
- *one_to_many* will ensure that the join key in the **left table is unique**
- *many_to_one* will ensure that the join key in the **right table is unique**

```
# use the validate argument to check unique key
df_mascots2= df_mascots.rename(columns={'HOGWARTS_HOUSE': 'HOUSE'})
pd.merge(df_hp_students,
        df_mascots2,
        on = 'HOUSE',
        how='left',
        validate='many_to_one')
```

✓ 0.0s

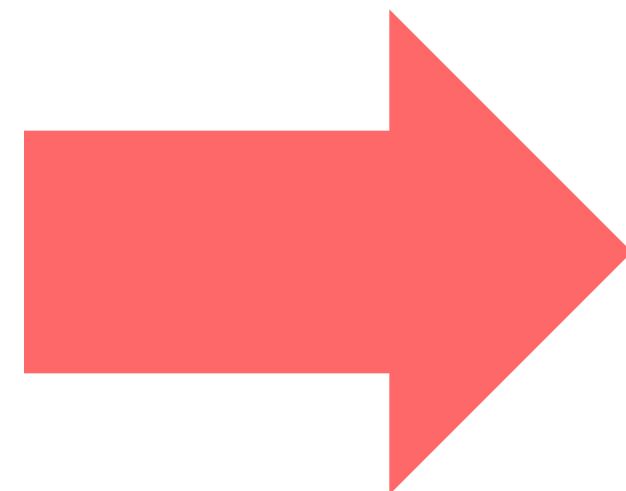
	NAME	HOUSE	ANIMAL	SOURCE
0	Harry Potter	Gryffindor	Lion	mascots_1
1	Ron Weasley	Gryffindor	Lion	mascots_1
2	Luna Lovegood	Ravenclaw	Eagle	mascots_2

Select unique rows

Pandas **drop_duplicates()** method helps in removing duplicates from the a Dataframe

	a	b	c
0	4	2	2
1	4	2	2
2	3	6	9
3	8	8	9

`df.drop_duplicates()`



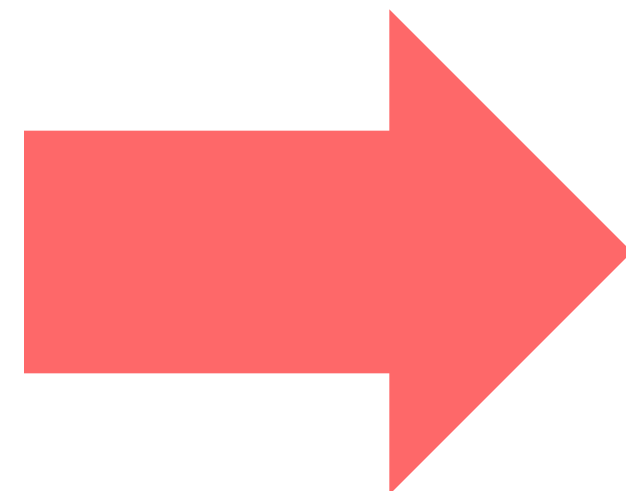
	a	b	c
0	4	2	2
2	3	6	9
3	8	8	9

Keep the last instance

Use the *keep* keyword to indicate to select last duplicate record

	a	b	c
0	4	2	2
1	4	2	2
2	3	6	9
3	8	8	9

```
df.drop_duplicates(keep='last')
```



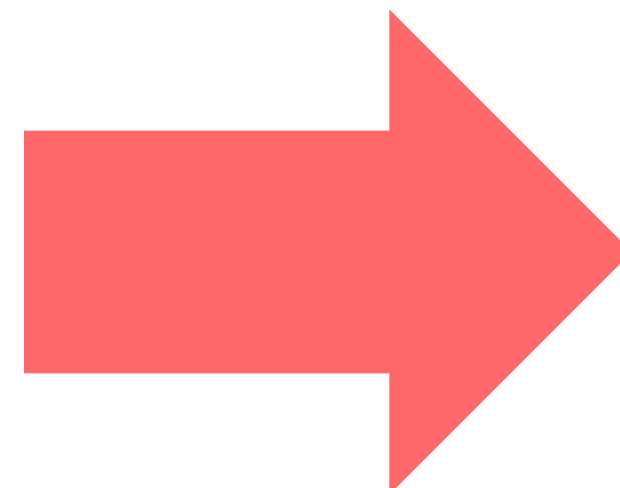
	a	b	c
1	4	2	2
2	3	6	9
3	8	8	9

Or select unique based on a column

Use the subset keyword to select unique rows to that specific column (default is to keep the first row)

	a	b	c
0	4	2	2
1	4	2	2
2	3	6	9
3	8	8	9

```
df_example.drop_duplicates(subset=['c'])
```



	a	b	c
0	4	2	2
2	3	6	9

We also have the `.join()` method

- Use the `.join()` method to combine data **on an index**
- While `merge()` is a module function, **`.join()` is a method that lives on your DataFrame**, which enables you to specify only one DataFrame, which will join the DataFrame you call `.join()` on.
- Under the hood, **`.join()` uses `merge()`**, but can be another way to join DataFrames

Use `set_index()` to specific join field

```
# The default for df.join is to perform a left join
# You need to set the index to join on
df_hogwarts = df_hp_students.set_index('HOUSE').join(df_mascots.set_index('HOGWARTS_HOUSE'))
df_hogwarts
```

✓ 0.0s

	NAME	ANIMAL	SOURCE
Gryffindor	Harry Potter	Lion	mascots_1
Gryffindor	Ron Weasley	Lion	mascots_1
Ravenclaw	Luna Lovegood	Eagle	mascots_2

House is now the index

Use “how” for other join types

```
# Other join types can be specified with how argument
```

```
df_hogwarts = df_hp_students.set_index('HOUSE').join(df_mascots.set_index('HOGWARTS_HOUSE'), how = "outer")
```

```
df_hogwarts
```

✓ 0.0s

	NAME	ANIMAL	SOURCE
Gryffindor	Harry Potter	Lion	mascots_1
Gryffindor	Ron Weasley	Lion	mascots_1
Hufflepuff	NaN	Badger	mascots_2
Ravenclaw	Luna Lovegood	Eagle	mascots_2
Slytherin	NaN	Snake	mascots_1

Let's take a quick break

We'll resume in

8 minutes...

Let's take a quick break

We'll resume in

7 minutes...

Let's take a quick break

We'll resume in

6 minutes...

Let's take a quick break

We'll resume in

5 minutes...

Let's take a quick break

We'll resume in

4 minutes...

Let's take a quick break

We'll resume in

3 minutes...

Let's take a quick break

We'll resume in
2 minutes...

Let's take a quick break

We'll resume in

1 minute...

Let's consider an example

A transportation company has these datasets:

1. **drivers** lists every driver's *driver_id*, *driver_name*, and *driver_phone*.
2. **routes** lists every route's *route_id*, *route_name*, and *route_miles*
3. **trips** lists every driver that drove last month, their *driver_id*, the *datetime*, and a *route_id* indicating where they drove

Which join to use?

On average, how many miles did *drivers who drove* drive?

1. **drivers** lists every driver's *driver_id*, *driver_name*, and *driver_phone*.
2. **routes** lists every route's *route_id*, *route_name*, and *route_miles*
3. **trips** lists every driver that drove last month, their *driver_id*, the *datetime*, and a *route_id* indicating where they drove

Which join to use?

On average, how many miles did *drivers who drove* drive?

```
trip_miles = pd.merge(trips,  
                      routes,  
                      on = 'route_id',  
                      how = 'inner',  
                      validate = 'many_to_one')  
trip_miles.route_miles.mean()
```

Which join to use?

On average, how many miles did *all drivers* drive?

1. **drivers** lists every driver's *driver_id*, *driver_name*, and *driver_phone*.
2. **routes** lists every route's *route_id*, *route_name*, and *route_miles*
3. **trips** lists every driver that drove last month, their *driver_id*, the *datetime*, and a *route_id* indicating where they drove

Which join to use?

On average, how many miles did *all drivers* drive?

```
driver_miles = pd.merge(pd.merge(df_drivers,
                                df_trips,
                                on = 'driver_id',
                                how = 'left',
                                validate = 'one_to_many'),
                        df_routes, on = 'route_id', how = 'left', validate = 'many_to_one')
driver_miles = driver_miles.route_miles.fillna(0)
driver_miles.route_miles.mean()
```

Challenge: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least—include the user friendly names in the *df_restaurant* table.

Challenge: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least—include the user friendly names in the *df_restaurant* table.

We'll resume in
8 minutes...

Challenge: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least—include the user friendly names in the *df_restaurant* table.

We'll resume in
7 minutes...

Challenge: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least—include the user friendly names in the *df_restaurant* table.

We'll resume in
6 minutes...

Challenge: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least—include the user friendly names in the *df_restaurant* table.

We'll resume in
5 minutes...

Challenge: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least—include the user friendly names in the *df_restaurant* table.

We'll resume in
4 minutes...

Challenge: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least—include the user friendly names in the *df_restaurant* table.

We'll resume in
3 minutes...

Challenge: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least—include the user friendly names in the *df_restaurant* table.

We'll resume in
2 minutes...

Challenge: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least—include the user friendly names in the *df_restaurant* table.

We'll resume in
1 minute...

Challenge: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least—include the user friendly names in the *df_restaurant* table.

One possible solution

```
rest_orders_merge = pd.merge(df_orders
                             , df_restaurants
                             , on = "RESTAURANT_ID"
                             , how = "left"
                             , validate = "many_to_one")
colors = ['blue', 'grey', 'grey', 'grey', 'grey', 'grey', 'grey', 'orange']
rest_orders_merge.NAME.value_counts().plot(kind='barh', width=.8, color=colors)
sns.despine()
plt.gca().spines["left"].set_visible(False)
plt.tick_params(left = False)
plt.xlabel("# OF ORDERS", loc = "left")
plt.ylabel("RESTAURANT", loc="top", rotation=0)
plt.title("Number of orders by restaurant", loc="left", pad=50, fontsize=20)
```

✓ 0.9s

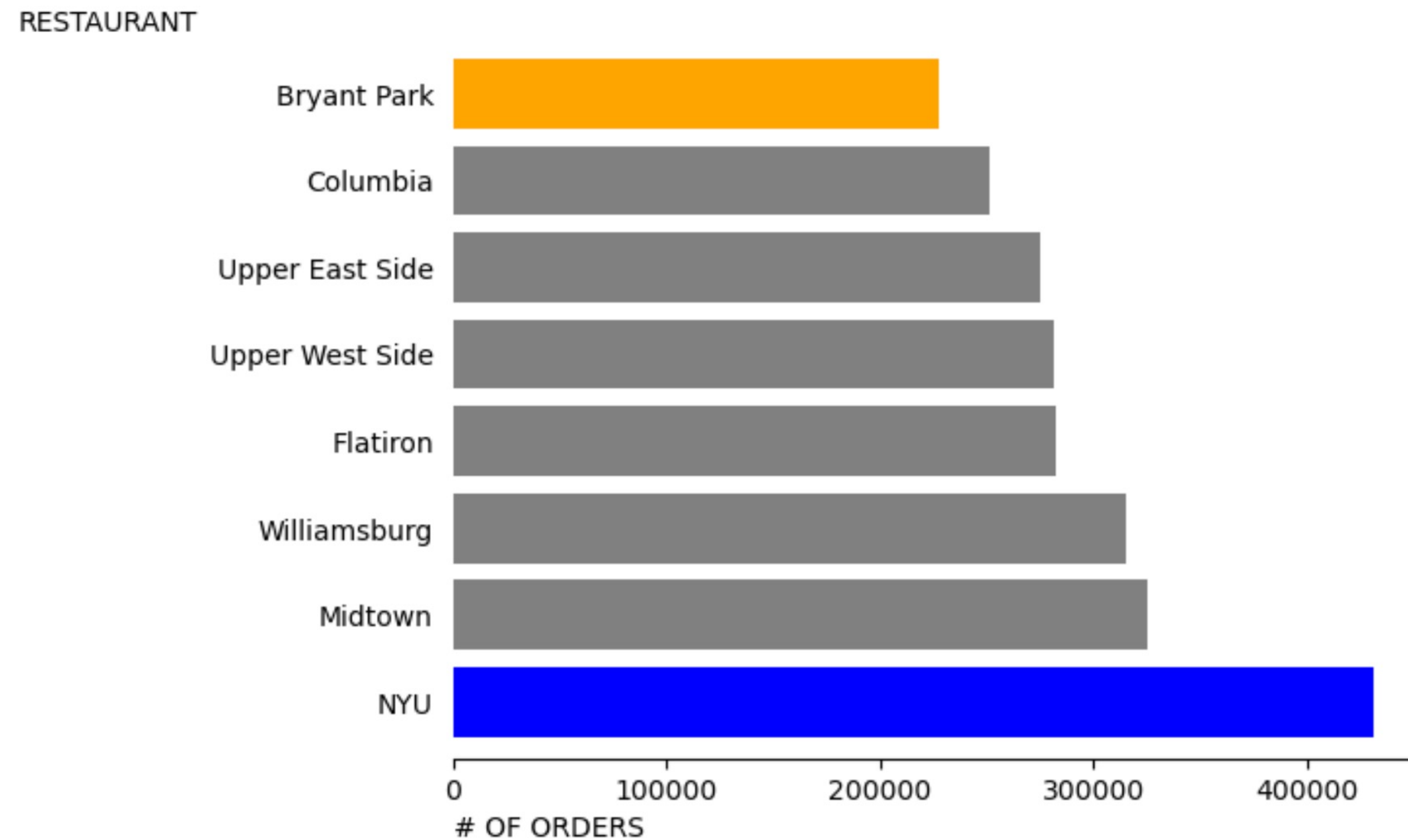
Another possible solution

```
df_orders.set_index("RESTAURANT_ID").join(df_restaurants.set_index("RESTAURANT_ID"))
colors = ['blue', 'grey', 'grey', 'grey', 'grey', 'grey', 'grey', 'orange']
rest_orders_merge.NAME.value_counts().plot(kind='barh', width=.8, color=colors)
sns.despine()
plt.gca().spines["left"].set_visible(False)
plt.tick_params(left = False)
plt.xlabel("# OF ORDERS", loc = "left")
plt.ylabel("RESTAURANT", loc="top", rotation=0)
plt.title("Number of orders by restaurant", loc="left", pad=50, fontsize=20)
```

✓ 1.2s

Both yield the same results

Number of orders by restaurant



Recap

concat() for combining DataFrames across rows or columns

- default is to add rows
- `axis = 1` adds columns

merge() for combining data on common columns or indices

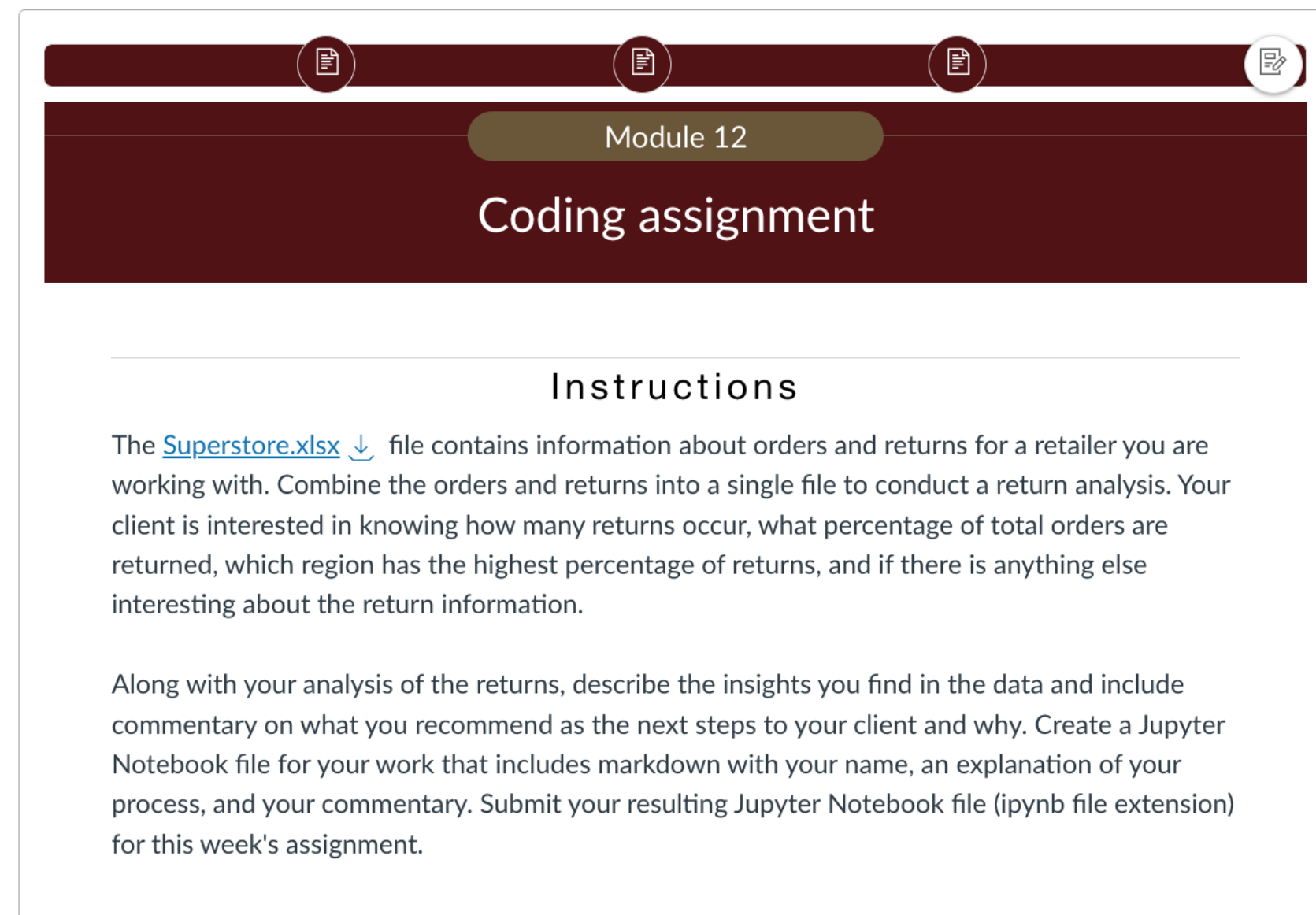
- *how* argument specifies join type
- *validate* argument check is keys are unique values

.join() for combining data on an index

- Use *set_index* to define key fields
- *how* argument specifies join type

Next week

Coding assignment due by next class



The screenshot shows a web interface for a course module. At the top, there is a dark red header bar with four circular icons. Below this, a light green bar contains the text "Module 12". Underneath, a dark red bar contains the text "Coding assignment". The main content area is white and titled "Instructions". It contains two paragraphs of text. The first paragraph mentions a file named "Superstore.xlsx" and describes the task of combining orders and returns data for analysis. The second paragraph provides additional instructions on what to include in the analysis and how to submit the work as a Jupyter Notebook file.

Module 12

Coding assignment

Instructions

The [Superstore.xlsx](#) file contains information about orders and returns for a retailer you are working with. Combine the orders and returns into a single file to conduct a return analysis. Your client is interested in knowing how many returns occur, what percentage of total orders are returned, which region has the highest percentage of returns, and if there is anything else interesting about the return information.

Along with your analysis of the returns, describe the insights you find in the data and include commentary on what you recommend as the next steps to your client and why. Create a Jupyter Notebook file for your work that includes markdown with your name, an explanation of your process, and your commentary. Submit your resulting Jupyter Notebook file (ipynb file extension) for this week's assignment.

Next week: **grouping and aggregating data** (I'll be in San Marcos)